

SparseLoom: Structured Sparsity for Long-Context Retrieval

Proceedings of the Workshop on Efficient Retrieval, 2023

Abstract

We present SparseLoom, a block-sparse attention approach to long-context passage retrieval. Existing systems treat this problem as a single monolithic decision, which wastes computation on inputs that are easy and starves the inputs that are hard. SparseLoom instead allocates effort where the evidence is ambiguous. On LoomBench we reach 61.4 nDCG@10, against 54.9 nDCG@10 for the DenseWeave baseline, while remaining trainable on a single 24GB consumer GPU.

1 Introduction

Work on long-context passage retrieval has converged on a small number of architectures that differ more in their training recipes than in their inductive biases. Progress reported on public leaderboards is therefore difficult to attribute, and practitioners choosing a system for a modest deployment have little guidance beyond parameter count. We take the opposite starting point and ask what can be achieved when the compute budget is fixed in advance at a single 24GB consumer GPU.

Our contribution is SparseLoom, which reorganises long-context passage retrieval around the observation that most inputs are decided by a small fraction of the available evidence. Rather than scaling the model, we make the decision procedure adaptive. This is a deliberately unfashionable direction: it trades headline capacity for predictable cost, and it is the trade practitioners actually face.

We evaluate on LoomBench, a collection of 48,000 examples assembled for long-context passage retrieval. Against the DenseWeave baseline, SparseLoom improves nDCG@10 from 54.9 to 61.4. An ablation shows that removing the block router costs 4.8 points of nDCG@10, which locates the gain in the component we claim is responsible rather than in incidental tuning.

The remainder of this paper is organised as follows. Section 2 describes SparseLoom in detail. Section 3 sets out the experimental protocol and the baselines we compare against. Section 4 reports results and ablations. Section 5 discusses the limitations of the approach, of which the most serious is that performance degrades on documents shorter than 200 tokens, where the block structure has too little material to exploit.

Prior work on long-context passage retrieval divides roughly into two camps. The first scales the encoder and accepts the resulting inference cost, on the assumption that capacity is the binding constraint. The second compresses an expensive model into a cheap one and accepts a fixed quality loss. Neither camp adapts the computation to the individual input, which is the gap SparseLoom occupies.

2 Method

SparseLoom has three stages: a cheap scorer that produces a provisional decision, a confidence estimator that decides whether the provisional decision is safe, and an expensive scorer that is invoked only when it is not. The three stages share an encoder, so the additional parameter cost over the

DenseWeave baseline is under four percent.

The confidence estimator is the component that makes the block-sparse attention framing pay. We train it with a margin objective against the disagreement between the cheap and expensive scorers, which requires no additional human annotation: the supervision is manufactured from the two scorers that already exist.

Training proceeds in two phases. The cheap and expensive scorers are trained jointly for the first phase, then frozen while the estimator is fitted. Freezing matters. When all three are trained end to end the estimator learns to mark everything as uncertain, because that minimises loss without ever being penalised for the compute it spends.

Inference is a single forward pass for the majority of inputs. On LoomBench the expensive scorer fires on roughly one input in five, which is what makes SparseLoom runnable on a single 24GB consumer GPU at all. We provide the exact thresholds and the schedule used for every reported number in the supplementary material.

The cheap scorer is a two-layer projection over pooled encoder states. We chose pooling over a learned attention head after finding that the two performed within noise of one another on validation, and pooling has no parameters to tune. Where two designs tie, we take the one with fewer knobs, because every knob is a future source of irreproducibility.

3 Experiments

We evaluate on LoomBench, which contains 48,000 examples. We use the standard split released with the dataset and report $nDCG@10$ on the held-out test portion. No test example is used for threshold selection; thresholds are chosen on a validation split carved out of the training portion.

Our primary baseline is DenseWeave, which we retrain under our own protocol rather than quoting published numbers. Retraining moves the DenseWeave figure by more than a point relative to the number reported by its authors, which is itself a small argument for retraining baselines rather than copying tables.

All runs use a single 24GB consumer GPU. Every reported figure is the mean of five seeds. We report the mean rather than the best seed because the seed-to-seed spread on LoomBench is roughly one point of $nDCG@10$, which is large enough that a best-of-five protocol would manufacture most of a claimed improvement.

We additionally run an ablation in which removing the block router is applied, holding every other setting fixed. This isolates the contribution of the component we claim is doing the work, and it is reported alongside the main result in the next section.

Hyperparameters were selected by random search over sixteen configurations on the validation split, with the search budget fixed in advance and identical for SparseLoom and for DenseWeave. Equalising the search budget matters more than the search method: an unequal budget is the most common way a comparison on LoomBench becomes unfair without anyone intending it.

4 Results

SparseLoom reaches 61.4 $nDCG@10$ on LoomBench. The DenseWeave baseline reaches 54.9 $nDCG@10$ under the identical protocol. The gap is stable across all five seeds and does not depend on the choice of validation split.

The ablation confirms where the gain comes from. With removing the block router, nDCG@10 falls by 4.8 points, recovering most of the distance back to DenseWeave. We read this as evidence that the improvement is a property of the block-sparse attention design rather than of the training schedule.

Cost is the other half of the claim. SparseLoom completes the full LoomBench test pass on a single 24GB consumer GPU, which the uncascaded configuration does not. For a team without cluster access this is the difference between a system that can be evaluated and one that can only be cited.

Breaking the LoomBench test set down by input length shows the gain is not uniform. The longest quartile of inputs accounts for most of the improvement, which is consistent with the block-sparse attention account: long inputs are where the cheap scorer is least reliable and where routing therefore has the most to decide.

5 Limitations

The most serious limitation of SparseLoom is that performance degrades on documents shorter than 200 tokens, where the block structure has too little material to exploit. We report this prominently because it determines whether the method is appropriate for a given deployment, and it is not visible from the headline nDCG@10 figure.

A second limitation is evaluation scope. LoomBench is a single dataset in a single language, and we make no claim that the 4.8 point ablation gap would reproduce elsewhere. Cross-dataset transfer is the obvious next study and we have not run it.

References

- [1] A. Renner and B. Okafor. Scaling laws for long-context passage retrieval. *Journal of Retrieval Systems*, 2021.
- [2] C. Lindqvist. DenseWeave: a strong baseline for long-context passage retrieval. In *Proceedings of the Conference on Information Access*, 2020.
- [3] D. Marchetti and E. Yilmaz. The LoomBench collection. Technical report, 2021.
- [4] F. Novak. On the reproducibility of leaderboard results. *Transactions on Empirical Methods*, 2022.
- [5] G. Adeyemi and H. Sorensen. Evaluation protocols that survive contact with practice. *Annual Review of Applied Machine Learning*, 2023.